

SmartHome IoT: Sistema de automatización y monitoreo para una casa inteligente

Universidad: Universidad de Talca
Asignatura: IoT (Internet Of Things)
Equipo: Equipo Hector
Integrantes:

- Alexis Ferghman
- Jeremy Iturriaga
- Héctor Vásquez

Profesor: Ricardo Pérez

Fecha: 9 de junio de 2026

1.- Introducción y descripción del problema

El Internet de las Cosas permite conectar dispositivos físicos a una red para monitorear variables del entorno, procesar información y ejecutar acciones automáticas o manuales. Una de sus aplicaciones más comunes es la automatización del hogar, donde distintos sensores y actuadores permiten mejorar la seguridad, el confort y la supervisión remota de una vivienda.

En este proyecto se desarrolló un prototipo de casa inteligente llamado **SmartHome IoT**, cuyo objetivo es monitorear variables ambientales y de seguridad dentro de un espacio cerrado. El sistema integra sensores de temperatura, humedad, gas inflamable, monóxido de carbono y sonido, además de una cámara ESP32-CAM para la visualización remota del entorno.

El problema consiste en la necesidad de contar con un sistema capaz de supervisar condiciones ambientales, detectar posibles situaciones anómalas y notificar al usuario en tiempo real. Por ejemplo, una temperatura elevada, un aumento en los valores de gas o un nivel de sonido anormal pueden representar eventos peligrosos para la seguridad del hogar.

Para resolver este problema, se implementó una arquitectura IoT basada en comunicación MQTT, una placa Arduino MKR1000 para la lectura de sensores, una ESP32-CAM para captura de imagen y Node-RED como plataforma de integración, visualización, procesamiento de datos y generación de alertas.

2.- Objetivo general y objetivos específicos

Objetivo general

Diseñar e implementar un prototipo IoT funcional para el monitoreo ambiental y de seguridad de una vivienda, integrando sensores físicos, comunicación MQTT, visualización en Node-RED Dashboard y notificaciones automáticas ante eventos críticos.

Objetivos específicos

1. Conectar y programar sensores de temperatura, humedad, gas inflamable, monóxido de carbono y sonido en una placa Arduino MKR1000.
2. Publicar las lecturas de los sensores hacia un broker MQTT usando una jerarquía ordenada de tópicos.
3. Diseñar un dashboard en Node-RED para visualizar en tiempo real las variables medidas por el sistema.
4. Implementar gráficos históricos para analizar la evolución de variables como temperatura, MQ6 y MQ7.
5. Configurar reglas de alerta basadas en umbrales para detectar condiciones anormales.
6. Enviar notificaciones externas mediante Telegram ante eventos críticos.

7. Registrar datos históricos en un archivo CSV desde Node-RED.
8. Integrar una ESP32-CAM para captura manual de imagen y visualización de eventos de reconocimiento.
9. Implementar reconocimiento facial mediante un servicio Python externo.
10. Implementar control manual de un actuador mediante un LED que representa la apertura y cierre de una puerta.
11. Implementar una acción automática que active el LED cuando se reconozca una persona conocida.
12. Documentar la arquitectura, decisiones técnicas, resultados y problemas encontrados durante el desarrollo.

3.- Diagrama de arquitectura del sistema

Para el funcionamiento de la casa inteligente se implementó un circuito similar al presentado en la [Figura 1](#). En dicho diagrama se muestran las conexiones principales de todos los sensores utilizados en el proyecto con la placa Arduino.

Debido a las limitaciones del software utilizado para diseñar la ilustración, fue necesario realizar algunas sustituciones respecto al circuito real. En primer lugar, los sensores MQ-6 y MQ-7 fueron representados mediante sensores MQ-2, ubicados en la parte inferior izquierda del diagrama. Esta sustitución no afecta la representación del circuito, ya que los tres sensores poseen una distribución de pines y un método de conexión equivalentes.

Asimismo, la placa Arduino MKR1000 fue reemplazada por un Arduino Nano, debido a la disponibilidad de componentes en la herramienta de diseño. Sin embargo, ambos dispositivos cuentan con pines de entrada y salida suficientes para representar adecuadamente las conexiones del proyecto. De manera similar, el sensor de temperatura y humedad AM2315C/AHT20 fue sustituido por un DHT22, ya que este último cumple una función equivalente dentro de la representación gráfica, permitiendo ilustrar correctamente la integración del sensor ambiental en el sistema.

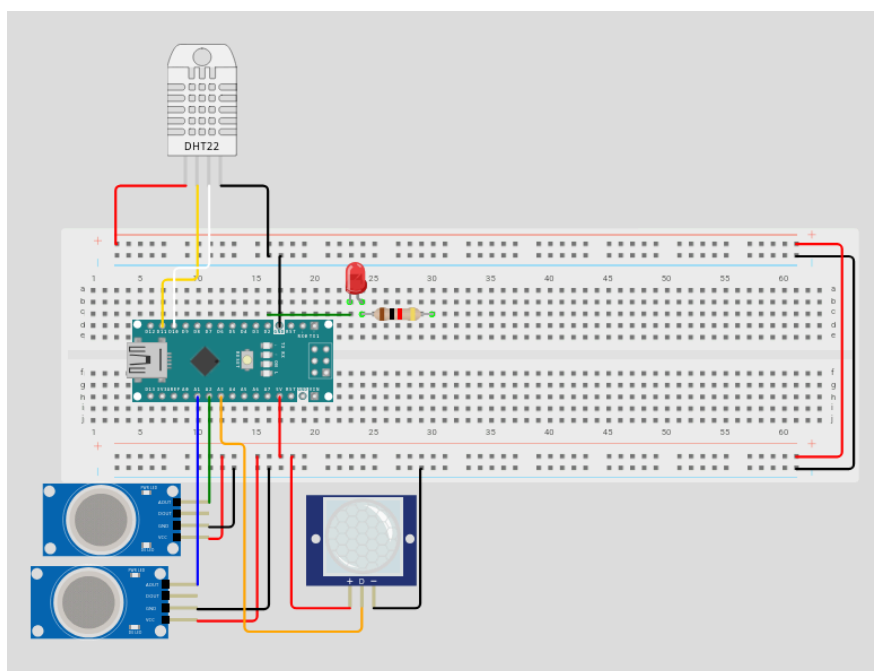


Figura 1. Diagrama de circuito MKR1000

4.- Descripción de sensores utilizados y rol en el sistema

Sensor de temperatura y humedad AM2315C / AHT20

Este sensor permite medir la temperatura ambiente y la humedad relativa del espacio monitoreado. Su función principal es entregar información ambiental que permita detectar condiciones anómalas dentro del hogar. La temperatura se expresa en grados Celsius, mientras que la humedad se expresa en porcentaje.

En el sistema, la temperatura se utiliza para generar alertas cuando supera un umbral definido, por ejemplo, 30 °C. La humedad permite observar las condiciones ambientales y detectar posibles niveles elevados que podrían afectar el confort o indicar problemas de ventilación.

Sensor MQ6

El sensor MQ6 se utiliza para detectar variaciones asociadas a gases inflamables, como gas licuado de petróleo. En este proyecto se emplea su salida analógica, obteniendo valores relativos entre 0 y 1023.

Su rol dentro del sistema es detectar aumentos anormales en la concentración de gas. Cuando el valor supera un umbral definido experimentalmente, Node-RED genera una alerta y envía una notificación por Telegram.

Sensor MQ7

El sensor MQ7 está orientado a la detección de monóxido de carbono. Al igual que el MQ6, en este proyecto se utiliza su lectura analógica como una referencia relativa, no como una medición certificada en partes por millón.

Su función es aportar una capa adicional de seguridad al sistema, permitiendo identificar posibles condiciones peligrosas asociadas a la presencia de monóxido de carbono.

Micrófono

El sensor de sonido permite detectar variaciones en el nivel de ruido del ambiente. En el sistema se utiliza para identificar eventos sonoros anormales, como golpes, ruidos fuertes o actividad inesperada.

Cuando el valor del micrófono supera un umbral definido, se genera una alerta que puede ser visualizada mediante Telegram.

ESP32-CAM

La ESP32-CAM permite capturar o transmitir imágenes del entorno monitoreado. Su rol dentro del sistema es entregar verificación visual remota, permitiendo al usuario observar el estado del lugar desde el dashboard de Node-RED y un reconocimiento de personas.

5.- Estructura de tópicos MQTT y ejemplos de mensajes JSON

Para la comunicación MQTT se definió una jerarquía de tópicos ordenada. Todos los tópicos comienzan con:

`smarthome/equipoHector`

Esto permite identificar que los mensajes pertenecen al sistema SmartHome del equipo Hector.

Tópicos de sensores

`smarthome/equipoHector/temperatura`

`smarthome/equipoHector/humedad`

`smarthome/equipoHector/gas/mq6`

`smarthome/equipoHector/gas/mq7`

`smarthome/equipoHector/sonido/microfono`

Tópico de control

smarthome/equipoHector/control/led

Tópico de cámara

smarthome/equipoHector/camara/evento

Calidad de servicio MQTT

En Node-RED se configuraron distintos niveles de QoS según la criticidad del dato. Las variables ambientales como temperatura, humedad y sonido se reciben con QoS 0, ya que se actualizan frecuentemente y no es crítico perder una lectura puntual. En cambio, los sensores MQ6 y MQ7 se configuraron con QoS 1, debido a que representan eventos de mayor relevancia para la seguridad. Los eventos de cámara se configuraron con QoS 2, ya que corresponden a eventos discretos de detección y reconocimiento.

Esta configuración permite diferenciar datos periódicos de eventos críticos y mejorar la confiabilidad de los mensajes más importantes.

Ejemplos de mensajes JSON

El Arduino MKR1000 publica cada variable como un JSON individual en su tópico correspondiente.

Ejemplo para temperatura:

Tópico:

smarthome/equipoHector/temperatura

```
{  
  "temperatura": 24.80  
}
```

Ejemplo para humedad:

Tópico:

smarthome/equipoHector/humedad

```
{  
  "humedad": 58.40  
}
```

Ejemplo para MQ6:

Tópico:

smarthome/equipoHector/gas/mq6

```
{  
  "mq6": 310  
}
```

Ejemplo para MQ7:

Tópico:

smarthome/equipoHector/gas/mq7

```
{  
  "mq7": 120  
}
```

Ejemplo para micrófono:

Tópico:

smarthome/equipoHector/sonido/microfono

```
{  
  "microfono": 740  
}
```

Ejemplo de evento de cámara con persona desconocida:

Tópico:

smarthome/equipoHector/camara/evento

```
{  
  "timestamp": "2026-06-05T17:46:17.236136+00:00",  
  "person_detected": true,  
  "known_person": false,  
  "person_name": null,  
  "confidence": 0.7345  
}
```

Ejemplo de evento de cámara con persona conocida:

```
{  
  "timestamp": "2026-06-05T17:46:17.236136+00:00",  
  "person_detected": true,  
  "known_person": true,  
  "person_name": "hector",  
  "confidence": 0.8125,  
  "face_image_b64": "/9j/4AAQSkZJRgABAQ..."  
}
```

Descripción de los campos del evento de cámara:

timestamp: fecha y hora del evento.

person_detected: indica si se detectó una persona o rostro.

known_person: indica si la persona fue reconocida.

person_name: nombre de la persona reconocida. Si no se reconoce, queda como null.

confidence: nivel de confianza del reconocimiento.

face_image_b64: imagen del rostro codificada en base64.

Para el control manual de la puerta se utiliza el tópicos:

smarthome/equipoHector/control/led

Los mensajes enviados son simples:

ON

para abrir la puerta, y:

OFF

para cerrarla.

Aunque el control se envía como texto simple, el tópicos identifica claramente el actuador controlado.

6.- Descripción del dashboard y capturas de pantalla

El dashboard fue desarrollado en Node-RED Dashboard y permite visualizar el estado del sistema en tiempo real. La interfaz se organiza en grupos para facilitar la lectura de la información.

Estado general del sistema

El dashboard incluye una tarjeta de estado general. Esta tarjeta resume los valores actuales de temperatura, humedad, MQ6, MQ7 y sonido. Además, muestra si el sistema se encuentra en estado **NORMAL** o **ALERTA**.

Cuando algún sensor supera su umbral definido, el estado general cambia a **ALERTA**. Si todos los sensores se encuentran dentro de los rangos normales, el estado se muestra como **NORMAL**.

 **ALERTAS ACTIVAS**

 **Alertas Activas**

 **Sin alertas activas**

Todos los sensores están dentro de rangos normales.

Última revisión: 09-06-2026, 10:13:07 a. m.



Figura 2. Tarjeta de estado general del sistema.

Visualización de sensores

El dashboard incluye gauges individuales para:

- Temperatura.
- Humedad.
- Gas MQ6.
- Gas MQ7.
- Sonido.

Cada gauge se actualiza en tiempo real (Cada 5s recibe una lectura para qué se actualice) a partir de los datos publicados por el Arduino MKR1000 mediante MQTT.



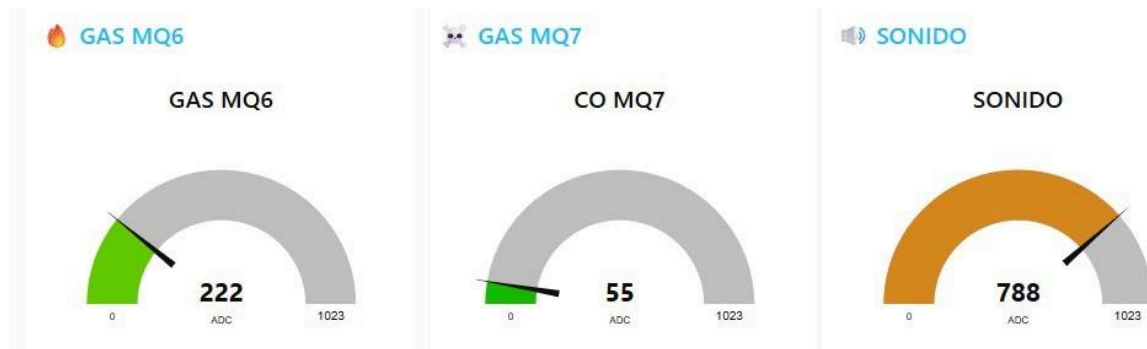


Figura 3. Gauges de temperatura, humedad, MQ6, MQ7 y sonido.

Gráficos históricos

Se implementaron gráficos históricos para observar la evolución temporal de variables relevantes. El sistema incluye historial de temperatura, MQ6 y MQ7. Estos gráficos permiten detectar tendencias o variaciones anómalas en el ambiente.

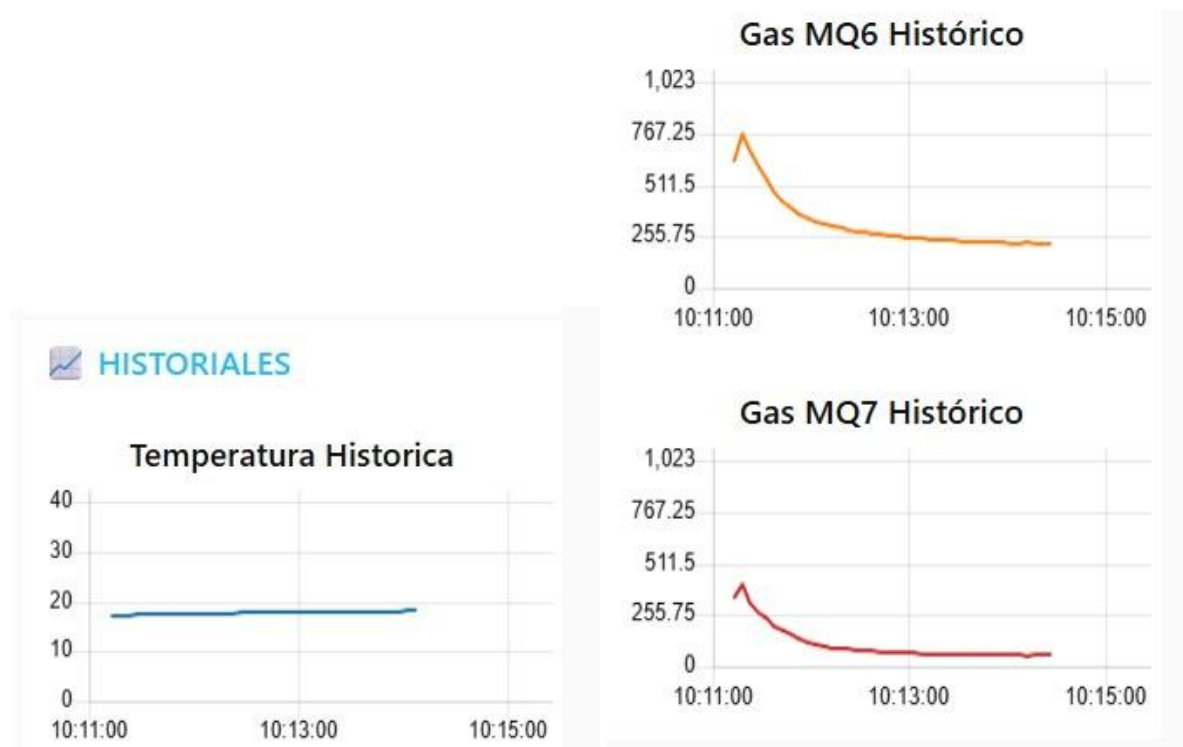


Figura 4. Gráficos históricos de temperatura, MQ6 y MQ7.

Cámara y reconocimiento facial

El dashboard incluye una sección de cámara donde se muestra el resultado del reconocimiento facial. Node-RED recibe eventos MQTT desde el servicio Python en el tópico:

smarthome/equipoHector/camara/evento

Cuando se detecta una persona, el mensaje recibido contiene información como el nombre reconocido, nivel de confianza, timestamp y una imagen del rostro codificada en base64. Node-RED procesa esta información y la muestra en una tarjeta visual del dashboard.

Si la persona es conocida, se muestra su nombre junto con un indicador positivo. Si no es reconocida, se muestra como persona desconocida.



Figura 5. Tarjeta de cámara con persona reconocida

Captura manual

También se incorporó un botón de captura manual. Al presionar el botón "Tomar Foto", Node-RED realiza una petición HTTP hacia la ESP32-CAM en la ruta **/capture**, recibe la imagen como buffer, la convierte a base64 y la muestra directamente en el dashboard.

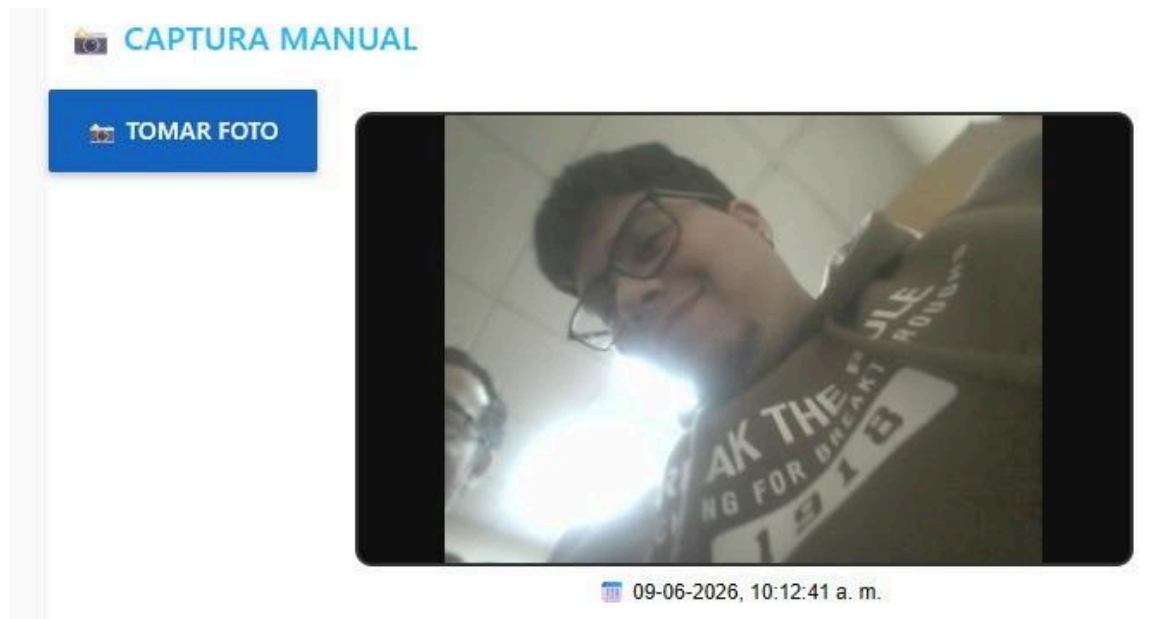


Figura 6. Botón “Tomar Foto” y resultado de captura manual.

Control manual de puerta

El dashboard incluye un switch llamado “Ábrete Sésamo”. Este switch permite controlar manualmente el LED conectado al Arduino MKR1000. Cuando el switch está activado, Node-RED publica **ON** en el tópico:

smarthome/equipoHector/control/led

Cuando se desactiva, publica **OFF** en el mismo tópico.

Este LED representa la apertura o cierre de una puerta.



Figura 7. Switch de control manual “Ábrete Sésamo”.

7.- Estrategias de control implementadas

El sistema implementa estrategias de control basadas en reglas de umbral, eventos de reconocimiento facial y acciones manuales desde el dashboard.

Temperatura alta

Si la temperatura supera los 30 °C, Node-RED genera una alerta de temperatura alta. Para evitar spam de mensajes, se utiliza memoria de contexto en el nodo function. Esto permite enviar una alerta solo cuando se supera el umbral por primera vez, cuando el valor sigue aumentando significativamente o cuando vuelve al estado normal.

La lógica considera tres estados principales:

1. Temperatura normal.
2. Temperatura sobre el umbral.
3. Temperatura normalizada.

Humedad alta

Si la humedad supera el 75 %, se genera una alerta de humedad alta. Al igual que en temperatura, se evita enviar mensajes repetidos mientras el valor se mantiene sobre el umbral. Además, se considera una variación mínima antes de enviar una nueva alerta de aumento.

Gas inflamable MQ6

Si el sensor MQ6 supera los 600 ADC, se genera una alerta de gas inflamable detectado. Además, se usa un umbral de normalización menor para evitar falsos cambios de estado. El sistema solo informa que el gas se disipó cuando el valor baja a un nivel seguro definido.

Esta lógica permite evitar oscilaciones de alerta cuando el valor se encuentra cerca del umbral.

Monóxido de carbono MQ7

Si el sensor MQ7 supera los 500 ADC, se genera una alerta asociada a posible presencia de monóxido de carbono. También se utiliza lógica de normalización para evitar cambios repetidos por pequeñas variaciones.

Ruido fuerte

Si el valor del micrófono supera los 900 ADC, se considera un evento de ruido fuerte. Mientras el ruido se mantiene alto, el sistema evita enviar notificaciones repetidas. Cuando el valor vuelve bajo el umbral de normalización, el sistema informa que el silencio fue restablecido.

Reconocimiento facial

Cuando se recibe un evento en:

smarthome/equipoHector/camara/evento

Node-RED verifica si se detectó una persona y si la confianza supera el umbral definido. Si la condición se cumple, se genera una notificación por Telegram y se actualiza el dashboard con información de la persona detectada.

En el caso de persona conocida, se muestra el nombre reconocido. En el caso de persona desconocida, se muestra el evento como detección no reconocida.

Encendido automático de LED por reconocimiento

El sistema también implementa una acción automática asociada al reconocimiento facial. Si la cámara detecta una persona conocida con una confianza mínima, Node-RED publica **ON** en el tópico del LED, encendiéndose. Luego, después de 5 segundos, publica **OFF** en el mismo tópico para apagarlo.

Esta acción simula la apertura automática de una puerta al reconocer a una persona autorizada.

Control manual de puerta

Además del control automático, el usuario puede controlar manualmente el LED desde el dashboard mediante el switch “Ábrete Sésamo”. Esto permite demostrar control remoto manual mediante MQTT.

Cuando el usuario activa el switch, Node-RED envía: ON

Cuando lo desactiva, Node-RED envía: OFF]

8.- Descripción de notificaciones y registro histórico

Notificaciones externas por Telegram

Las notificaciones externas se implementaron mediante un bot de Telegram integrado con Node-RED. Cuando una variable supera su umbral o cuando ocurre un evento de reconocimiento facial, Node-RED genera un mensaje con el tipo de evento, valor detectado y fecha/hora.

Las alertas son enviadas a un grupo de Telegram, permitiendo que todos los integrantes reciban los eventos en tiempo real.

Ejemplo de notificación por temperatura:



Alerta SmartHome IoT

Evento: Temperatura ALTA superó umbral

Valor: 31.20 °C

Hora: 05-06-2026, 14:35:12

Ejemplo de notificación de gas inflamable:

 Alerta SmartHome IoT

Evento: Gas Inflamable (MQ6) DETECTADO

Valor: 650 ADC

Hora: 05-06-2026, 14:40:21

Ejemplo de notificación de cámara:

 Alerta SmartHome IoT

Evento: Persona desconocida detectada

Nombre: No reconocida

Confianza: 73.45 %

Fecha y hora: 05-06-2026, 13:46:17

Para evitar saturación de mensajes, se incorporó un nodo **delay** y lógica de memoria mediante **context** en los nodos function. Esto permite que las alertas no se repitan constantemente mientras una variable se mantiene sobre el umbral.

Notificaciones visuales en dashboard

Además de Telegram, el dashboard incluye secciones visuales de estado general y alertas activas. Estas permiten identificar rápidamente si el sistema se encuentra en estado normal o si existe una condición anómala.

Registro histórico CSV

El sistema registra datos históricos en un archivo CSV desde Node-RED. Cada vez que se recibe una lectura de sensor, se genera un registro con fecha y datos recibidos.

El archivo se almacena en:

/usr/src/node-red/datos_sensores_V2.csv

El registro histórico permite revisar posteriormente los valores publicados por los sensores y analizar el comportamiento del sistema.

El flujo de registro utiliza:

MQTT in → function preparar datos → CSV → file

La información guardada incluye fecha y el contenido del dato recibido. Esto permite mantener trazabilidad de las lecturas recibidas desde los tópicos MQTT.

Gráficos históricos

Además del registro CSV, el dashboard incorpora gráficos históricos de temperatura, MQ6 y MQ7. Estos gráficos permiten observar la evolución temporal de las variables y detectar aumentos progresivos que podrían indicar condiciones anómalas.

9.- Problemas encontrados y soluciones aplicadas

Durante el desarrollo del proyecto se presentaron distintos problemas técnicos.

Lecturas variables en sensores MQ

Los sensores MQ6 y MQ7 entregan lecturas analógicas que pueden variar según el ambiente, el tiempo de calentamiento y la exposición al gas. Para resolver esto, se trabajó con valores relativos en ADC y se definieron umbrales experimentales para las alertas.

Además, se incorporaron umbrales de normalización menores al umbral de alerta. Esto evita que el sistema cambie constantemente entre estado normal y alerta cuando el valor está cerca del límite definido.

Exceso de notificaciones

Al inicio, cuando una variable superaba un umbral, el sistema podía enviar notificaciones repetidas continuamente. Para evitar esto, se incorporó lógica de memoria usando **context** en Node-RED. De esta forma, el sistema recuerda si ya se encuentra en estado de alerta y solo vuelve a notificar si el valor aumenta de forma importante o vuelve a la normalidad.

También se utilizó un nodo **delay** para limitar la frecuencia de salida hacia Telegram.

Organización de tópicos MQTT

Inicialmente era posible enviar todos los datos por un único tópico general. Sin embargo, para facilitar la visualización directa en Node-RED, se decidió usar tópicos separados para cada variable. Esta decisión permitió conectar cada sensor directamente a su gauge, gráfico, regla de alerta y registro.

Integración de cámara

La integración de la cámara requirió trabajar con eventos MQTT y capturas HTTP. Para mostrar imágenes en el dashboard, Node-RED recibe la imagen como buffer, la convierte a base64 y la inserta en una tarjeta visual mediante **ui_template**.

Reconocimiento facial fuera de la ESP32-CAM

Uno de los desafíos fue que la ESP32-CAM no tiene suficiente capacidad para ejecutar reconocimiento facial robusto en tiempo real con modelos más pesados. Para resolverlo, se decidió separar responsabilidades: la ESP32-CAM solo transmite o entrega imágenes y el reconocimiento se ejecuta en un servicio Python externo.

Procesamiento de imagen en Python

El servicio Python debe leer imágenes de la cámara, detectar rostros, generar embeddings y comparar contra una base de personas conocidas. Para esto se utilizaron OpenCV y DeepFace con el modelo **ArcFace**. Esta solución permitió realizar reconocimiento facial sin sobrecargar la cámara.

Eventos repetidos de cámara

El reconocimiento facial puede detectar a la misma persona repetidamente en frames consecutivos. Para evitar publicar demasiados eventos MQTT, el sistema considera umbrales de confianza y lógica de control para filtrar detecciones.

Control manual y automático del LED

El LED se utilizó como actuador representativo de una puerta. Se implementó control manual desde el dashboard y control automático por reconocimiento de persona. Esto permitió cubrir tanto interacción manual como respuesta automática del sistema.

10. Conclusiones

El proyecto permitió implementar un prototipo funcional de casa inteligente basado en IoT, integrando sensores físicos, comunicación MQTT, Node-RED Dashboard, Telegram, registro histórico, cámara ESP32-CAM y reconocimiento facial.

Se logró medir temperatura, humedad, gas inflamable, posible monóxido de carbono y sonido mediante una placa Arduino MKR1000. Cada variable fue publicada en formato JSON individual por tópicos MQTT ordenados, permitiendo una integración clara con Node-RED.

El dashboard desarrollado permite visualizar los datos en tiempo real mediante gauges, revisar el estado general del sistema, observar alertas activas, analizar gráficos históricos y controlar manualmente un actuador. Además, se integró una sección de cámara con captura manual y visualización de eventos de reconocimiento facial.

Las reglas de control implementadas permiten detectar condiciones anormales y responder mediante notificaciones externas por Telegram. También se incorporó una acción automática que enciende el LED cuando se reconoce una persona conocida, simulando la apertura de una puerta automática.

La integración de la ESP32-CAM con un servicio Python permitió extender el proyecto más allá de un monitoreo ambiental básico. Al separar la captura de imagen del procesamiento facial, el sistema logró incorporar reconocimiento de personas sin sobrecargar el microcontrolador. Esto demuestra una arquitectura IoT distribuida, donde distintos nodos cumplen funciones específicas y se comunican mediante MQTT y HTTP.

Como mejora futura, el sistema podría incorporar una base de datos más robusta, como SQLite o InfluxDB, ampliar la cantidad de actuadores físicos, mejorar la calibración de los sensores MQ y agregar modelos de reconocimiento más precisos para la ESP32-CAM.

En conclusión, el sistema cumple con los objetivos principales de un prototipo de casa Inteligente, ya que permite monitorear el ambiente, detectar eventos relevantes, notificar al usuario, registrar datos y ejecutar acciones manuales y automáticas.